

## Project 2 – Protocols: SSH Key Exchange and Authentication

Benjamin Deutsch (12215881)

TU Wien

### Project Task 6.1: Proving that the Subprotocol is Well-Typed

### Project Task 6.2: Discussion

**T in the new typing rule for tuples/pairs and how to type  $\llbracket sid, n'_s \rrbracket_{sk_C}$ .** I believe that the type  $T$  in the new typing rule for tuples should be the smallest type  $T \in \{LH, LL, HH, HL\}$  (w.r.t.  $\sqsubseteq$ ) such that  $\forall 1 \leq i \leq n : T_{M_i} \leq T$ , where  $T_{M_i}$  is the type of  $M_i$ .

*Reason:* This enforces that if one of the  $M_i$  has a high confidentiality/low integrity level, then the whole tuple will also have a high confidentiality/low integrity level.

We could then type  $\llbracket sid, n'_s \rrbracket_{sk_C}$  as follows:

Since  $n'_s$  is generated by the server and is given the type  $HH$ , we can type  $n'_s$  with type  $HH$ . By assumption,  $\Gamma \vdash sid : LH$ . Therefore, by the new typing rule for tuples, we can type  $sid, n'_s$  with the type  $HH$ . Then, since we have  $\Gamma \vdash sk_C : \text{SigK}^{HH}[\![HH]\!]$  and a proof of  $sid, n'_s : HH$ , we can use the typing rule for signatures to type  $\llbracket sid, n'_s \rrbracket_{sk_C}$  with the type  $HH$ .

**Would the server process still typecheck if we remove the equality check on the verification key?** No, the server code would not typecheck if we remove the equality check. Since  $vk_C$  is decrypted using a key of type  $\text{SymK}^{HH}[\![HH]\!]$ ,  $vk_C$  gets the type  $HH$ . As verification keys must have low confidentiality, in order to typecheck, we would have to show that  $vk_C : LH$  or  $vk_C : LL$ , using subtyping. However, since  $HH \not\sqsubseteq LH$  and  $HH \not\sqsubseteq LL$ , we cannot type  $vk_C : LH$  or  $vk_C : LL$  and thus the server code would not typecheck.

Verification keys must have low confidentiality because they are public information. But, even though  $vk_C$  should be public, the fact that it arrived through a high confidentiality channel makes the typesystem treat it as a secret. By the security lattice, you cannot declare a high confidentiality value to be of low confidentiality. Hence, the server can't use verification keys from high confidentiality channels directly. The equality check resolves this issue, by replacing  $vk_C$  with an equal key of low confidentiality.

### Project Task 8.1: Warmup – Authentication Subprotocol

**How I represented the shared secret  $g^{ab}$  and the secure channel it provides.** Since in this exercise, we assume that the Diffie-Hellman key exchange is secure, I created a new secret  $gab$  of type *key* for each session to represent the shared secret  $g^{ab}$ .  $gab$  is then given to the client and the server. The channel over which the client and server communicate is public, but the messages are encrypted with  $gab$ . This is how I represent the secure channel provided by  $g^{ab}$ .

**My choices regarding parallelism and multiple sessions.** My process creates arbitrarily many clients with unique signing keys. Then, for each client, it creates arbitrarily many sessions with unique sessions keys. This way, we model multiple clients with multiple sessions in parallel. In the actual protocol, the session id  $sid$  is  $h(g^a, g^b)$ . Since in this exercise, we don't model the Diffie-Hellman key exchange, I created a function that takes the session key as input and outputs the session id. Since the session id and the verification key of the client are public knowledge, they are both sent to the  $att$  channel, which the attacker can observe.

**More details about my modeling choices.** I first modeled the nonce as it is in the protocol, i.e. the server generates a new nonce for each session. However, as far as I know, ProVerif does not support secrecy queries for local variables. So I had to create a global private variable for the nonce, which the server uses for its challenge instead of generating a new nonces. This allowed me to query the secrecy of the nonce. Since even in this setting, where the server uses the same nonce for every session, the nonce remains private, the fresh nonces in the actual protocol also remain private.

## Project Task 8.2: The Full Protocol

**Which queries I believe are relevant to the full protocol.** I believe that it is important that after the protocol execution, both the client and the server know that they are talking to the intended party. Therefore, I added the following queries:

- After the key exchange, the client knows they are talking to the server:

$$\begin{aligned} \forall skC : \text{sign\_key}, skS : \text{sign\_key} : & \text{event}(\text{clientKeyExchanged}(skC, vk(skS))) \\ \implies & \text{inj-event}(\text{serverBegin}(skS)) \end{aligned}$$

- After the server accepts, the server knows they are talking to the client:

$$\begin{aligned} \forall skS : \text{sign\_key}, skC : \text{sign\_key} : & \text{event}(\text{serverAccept}(skS, vk(skC))) \\ \implies & \text{inj-event}(\text{clientBegin}(skC)) \end{aligned}$$

Furthermore, I also added a message that is sent by the client to the server after the protocol execution, which is encrypted with the shared secret  $gab$ . I then query the secrecy of this message. This is to check that encrypted messages are secure.

**Limited parallelism** Ideally, we would like an arbitrary number of clients having sessions with arbitrary many servers in parallel. This could be expressed using the following process:

$$\begin{aligned} & !(\text{new } skC : \text{sign\_key}; \text{out}(att, vk(skC)); !\text{Client}(skC)) \mid \\ & !(\text{new } skS : \text{sign\_key}; \text{out}(att, vk(skS)); !\text{Server}(skS)) \end{aligned}$$

However, ProVerif is not able to prove that, after the key exchange, the client knows they are talking to the server, for this process. Therefore, I limited the parallelism by allowing unlimited clients, but only one server.

**Man-in-the-middle attack on the variant where client does not validate server's host key.** If the client does not validate the server's host key, then an attacker could completely impersonate any server to the client. The attacker could do this by generating their own signing key and following the protocol using their own signing and validation keys instead of the server's keys. Since the client does not check whether the received key is the expected one, the client would accept the attacker's verification key and thus establish a shared secret with the attacker instead of the server.

To be honest, I am not sure why this counts as a man-in-the-middle attack, since the attacker does not need to interact with the server at all. But the attacker could, of course, also establish an SSH session with the server as a client and then relay messages between the actual client and the server, impersonating the server to the client and impersonating the client to the server. This would allow the attacker to read and modify messages between the client and the server.

**How the queries are affected.** The only query that can still be proven in this setting is

$$\begin{aligned} \forall skS : \text{sign\_key}, skC : \text{sign\_key} : & \text{event}(\text{serverAccept}(skS, vk(skC))) \\ \implies & \text{inj-event}(\text{clientBegin}(skC)). \end{aligned}$$

The query

$$\begin{aligned} \forall skC : \text{sign\_key}, skS : \text{sign\_key} : & \text{event}(\text{clientKeyExchanged}(skC, vk(skS))) \\ \implies & \text{inj-event}(\text{serverBegin}(skS)) \end{aligned}$$

cannot be proven since it is actually false. As the attacker can impersonate the server to the client, the client could exchange keys with the attacker, without the server ever being involved.

Furthermore, the secrecy of the message encrypted with *gab* after the protocol execution cannot be proven either, since it is also not true. As the attacker can impersonate the server to the client, the attacker can establish a shared secret with the client and thus decrypt the message encrypted with their shared secret.

Interestingly, the queries checking whether the client and server can be executed, i.e. whether the end of the protocol can be reached, cannot be proven. However, if both parties are honest, then the protocol can still be executed and the corresponding events can be triggered.

**Fixing the attack.** To fix the attack, the client needs to check that the received verification key is the expected one, like it is done in `ssh_full.pv`. After this fix, all queries can be proven again.

**Forward secrecy.** Yes, the secrets protected under *gab* are forward secret. This is because *gab* is a fresh key that is generated for each session. The server's signing key does not reveal anything about *gab*, since *gab* is not derived from the server's signing key. Therefore, even if the server's signing key is compromised, past messages encrypted with *gab* are still secret.

To verify this property in ProVerif, I modified the process: Now there are two phases. In the first phase, arbitrarily many clients can interact with arbitrarily many servers in parallel (since we are not interested in the authentication properties here, we can allow for more parallelism). In the second phase, the attacker learns the server's signing key. As in the previous exercises, I made the client send a message encrypted with *gab* to the server after the protocol execution and then I queried the secrecy of this message. The query `not attacker(message)` is true, which means that the message is still secret, even after the server's signing key is leaked. Therefore, past messages encrypted with *gab* are still secret, even if the server's signing key is compromised. Thus, the protocol provides forward secrecy.